

Competitive Programming

Section 52 Editorial: Game Theory

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Nim Winner	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. Subtraction Game	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
4	C. DAG Game	7
4.1	Restatement	7
4.2	Observations and algorithm	7
4.3	Correctness proof	7
4.4	Complexity	7
4.5	Full C++ solution	7
4.6	Full Python solution	7
5	D. Multi Subtraction	9
5.1	Restatement and traps	9
5.2	One-pile Grundy values	9
5.3	Combining piles and trace	9
5.4	Algorithm	9
5.5	Correctness proof	10
5.6	Complexity and implementation notes	10
5.7	Full C++ solution	10
5.8	Full Python solution	11
5.9	Tests that matter	12
6	What to carry forward	13
7	Complete upsolve notes	14
7.1	A. Nim Winner	14
7.2	B. Subtraction Game	14
7.3	C. DAG Game	14

1 Editorial overview

1.1 Problem-specific proof map

A. Nim Winner. Nim xor zero is losing because every move makes it nonzero; nonzero xor has a move that lowers one heap and restores zero. This gives the winner in linear time per position.

B. Subtraction Game. $\text{win}[x]$ is true exactly when some allowed move reaches a losing $\text{win}[x-d]$; otherwise every legal move helps the opponent. Induction on x proves the recurrence.

C. DAG Game. A vertex's Grundy value is the mex of successor values. A topological reverse order ensures all successors are known, and Grundy zero is exactly a losing state.

1.2 Detailed solution guide

A. Nim XOR. Let x be the xor of all pile sizes. If $x = 0$, every move changes one pile and makes the xor nonzero, because changing one operand of an xor-zero collection cannot keep the xor zero. If $x \neq 0$, choose a pile whose highest set bit matches the highest set bit of x , and reduce it to $\text{pile} \text{ xor } x$, which is smaller. The new global xor is zero. Thus xor-zero positions are losing and nonzero positions are winning.

B. Subtraction Game DP. Let $\text{win}[i]$ mean the current player wins with i stones. The base $\text{win}[0] = \text{false}$ because no move is possible. For $i > 0$, $\text{win}[i]$ is true if there exists an allowed move $d \leq i$ with $\text{win}[i-d] == \text{false}$; otherwise every move gives the opponent a winning state, so $\text{win}[i]$ is false. This is induction on the number of stones.

C. Grundy on a DAG. A terminal node has Grundy value 0. For every other node, compute the mex of the Grundy values of its outgoing neighbors. In a DAG, reverse topological order guarantees children are known before parents. A start node is winning iff its Grundy value is nonzero, by the same mex argument used for impartial games in the lesson.

Common pitfalls. Do not confuse “has a move” with “is winning”; a move must go to a losing state. For mex, only reachable child values matter. For Nim, the winning move reduces one pile, never increases it.

Every problem is a normal-play impartial game.

- A uses the Nim theorem: xor zero is losing, xor nonzero is winning.
- B computes $\text{win}[i]$ for a subtraction game from smaller piles to larger piles.
- C computes Grundy numbers on a DAG by taking the mex of successor Grundy numbers.

The shared proof is recursive: a state is winning if it has at least one move to a losing state; it is losing if every move goes to a winning state.

2 A. Nim Winner

2.1 Restatement

For each Nim position, print WIN or LOSE.

2.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

2.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

2.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int T;if(!(cin>
```

2.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    t=data[0]; idx=1; out=[]
    for _ in range(t):
        n=data[idx]; idx+=1; x=0
        for v in data[idx:idx+n]: x^=v
```

```
        idx+=n; out.append("WIN" if x else "LOSE")
    print("\n".join(out))
if __name__=="__main__": main()
```

3 B. Subtraction Game

3.1 Restatement

Given total stones and allowed moves, print whether the first player wins.

3.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

3.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

3.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(cin>>n>>m))return 0;vector<int> moves(m);for(int i=0;i<m;i++)cin>>moves[i];vector<bool> win(n+1,0);for(int i=1;i<=n;i++){for(int j=0;j<m;j++){if(i-moves[j]>0)win[i]=win[i-moves[j]]||!win[i];}}cout<<(win[n]?1:0)<<endl;return 0;}
```

3.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; moves=data[2:]; win=[False]*(n+1)
    for i in range(1,n+1): win[i]=any(i>=x and not win[i-x] for x in moves)
    print("WIN" if win[n] else "LOSE")
if __name__=="__main__": main()
```

4 C. DAG Game

4.1 Restatement

For each start vertex in a DAG, print whether it is winning.

4.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

4.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

4.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m,q;if(!
```

4.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m,q=data[0],data[1],data[2]; idx=3; g=[] for _ in range(n)
    for _ in range(m): g[data[idx]-1].append(data[idx+1]-1); idx+=2
    gr=[0]*n
    for u in range(n-1,-1,-1):
```

```
    seen={gr[v] for v in g[u]}; x=0
    while x in seen: x+=1
    gr[u]=x
    out=["WIN" if gr[data[idx+i]-1] else "LOSE" for i in range(q)]
    print("\n".join(out))
if __name__=="__main__": main()
```

5 D. Multi Subtraction

5.1 Restatement and traps

There are several piles, and one turn changes exactly one pile by subtracting an allowed amount. The game ends only when every pile has no legal subtraction. This is not a single subtraction game with the pile sizes added together: a move in one pile does not alter the legal moves in another pile.

The largest pile size is at most 200000 and there are at most twenty move sizes. Simulating play on all combinations of pile sizes is impossible, but one pile has only `maximum_pile + 1` states.

5.2 One-pile Grundy values

Let $g[x]$ be the Grundy value of one pile containing x stones. Its moves go to the states $x-d$ for allowed $d \leq x$, so the defining recurrence is

```
g[0] = 0
g[x] = mex({ g[x-d] : d is allowed and d <= x })
```

Compute these values from zero through the largest input pile. At most m values are reachable from a state, so the mex is at most m ; a small boolean array or set is enough.

For moves $\{1,3,4\}$, the first values are:

x	0	1	2	3	4	5	6	7
$g[x]$	0	1	0	1	2	3	2	0

5.3 Combining piles and trace

The piles are independent impartial games. Their combined Grundy value is the xor of their individual values. The initial position is losing exactly when that xor is zero.

In the sample, the table gives $g[2]=0$, $g[4]=2$, and $g[7]=0$. The total is $0 \text{ xor } 2 \text{ xor } 0 = 2$, so the first player has a winning move and the answer is WIN.

5.4 Algorithm

```

read every pile and let H be the largest size
for x from 1 through H:
    g[x] = mex of g[x-d] over legal allowed moves d
xor g[pile] for every input pile
print WIN iff the xor is nonzero

```

5.5 Correctness proof

Lemma 1. For every pile size x , the computed value $g[x]$ is its Grundy number in the one-pile subtraction game.

Proof. $g[0]=0$ because no move is possible. Every move from x reaches a smaller pile $x-d$, whose Grundy number has already been computed correctly. Taking the mex of exactly those reachable values is the definition of the Grundy number. Induction on x proves the lemma.

Lemma 2. The xor of the computed Grundy values is the Grundy value of the whole multi-pile position.

Proof. A turn chooses one pile and leaves all other piles unchanged, so the position is the disjoint sum of its one-pile games. The Sprague-Grundy xor rule states that the Grundy value of such a sum is the xor of the component values.

Theorem. The algorithm prints WIN exactly when the first player can force a win.

Proof. By Lemma 1 the values used for every pile are correct, and by Lemma 2 their xor is the combined Grundy value. An impartial normal-play position is losing exactly when its Grundy value is zero, so the reported condition is exactly the winning condition.

5.6 Complexity and implementation notes

Let H be the largest input pile and m the number of moves. The DP takes $O(H \cdot m)$ time, and reading/xoring piles takes $O(p)$. Memory is $O(H)$. Do not compute only winning/losing values: xor needs the full Grundy numbers. Zero-size piles contribute $g[0]=0$ and therefore change nothing.

5.7 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int move_count, pile_count;
    if (!(cin >> move_count >> pile_count)) return 0;
    vector<int> moves(move_count), piles(pile_count);
    for (int& move : moves) cin >> move;

```

```

int maximum = 0;
for (int& pile : piles) {
    cin >> pile;
    maximum = max(maximum, pile);
}

vector<int> grundy(maximum + 1);
for (int stones = 1; stones <= maximum; ++stones) {
    bool seen[32]{};
    for (int move : moves) {
        if (move <= stones) seen[grundy[stones - move]] = true;
    }
    while (seen[grundy[stones]]) ++grundy[stones];
}
int total = 0;
for (int pile : piles) total ^= grundy[pile];
cout << (total ? "WIN" : "LOSE") << '\n';
}

```

5.8 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    move_count, pile_count = data[0], data[1]
    moves = data[2 : 2 + move_count]
    piles = data[2 + move_count : 2 + move_count + pile_count]
    maximum = max(piles)
    grundy = [0] * (maximum + 1)
    for stones in range(1, maximum + 1):
        reachable = {grundy[stones - move] for move in moves if move <= stones}
        while grundy[stones] in reachable:
            grundy[stones] += 1
    total = 0
    for stones in piles:
        total ^= grundy[stones]
    print("WIN" if total else "LOSE")

if __name__ == "__main__":
    main()

```

5.9 Tests that matter

```
All piles empty.  
One pile with no legal move.  
Two equal piles, whose equal Grundy values xor to zero.  
Move sets that omit 1, so early pile sizes can remain losing.  
Many piles with a small maximum size and few piles with a large maximum size.
```

The tiny random oracle runs direct minimax on the complete sorted tuple of pile sizes, rather than reusing Grundy values. The scale case has 200000 piles and uses the direct parity property of the one-move set {1}.

6 What to carry forward

7 Complete upsolve notes

7.1 A. Nim Winner

XOR all pile sizes. A zero XOR is losing: every move makes it nonzero. A nonzero XOR has a move that clears its highest set bit by reducing one pile, making XOR zero. This is $O(n)$, uses an integer XOR accumulator, and should test empty-equivalent zero piles, equal piles, and one pile.

7.2 B. Subtraction Game

Let $\text{win}[s]$ mean the player to move with s stones can force a win. It is true iff some legal subtraction reaches a losing state; $\text{win}[0]=\text{false}$. Fill upward through the target. The recurrence is an induction proof, costs $O(\text{target} * \text{moves})$, and must ignore moves larger than s . Test target zero, move one, and a move set with gaps.

7.3 C. DAG Game

Process vertices in reverse topological order. A vertex is winning iff it has an edge to a losing successor; sinks are losing. Every successor is already classified in this order, proving the recurrence. Complexity is $O(V+E)$; test sinks, disconnected components, and multiple query starts. Use iterative topological processing in Python for deep graphs.

Run `CP_TARGET=solution python3 sections/52_game_theory/check.py` and compare small subtraction/DAG states with a recursive minimax oracle.

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.